BALAJI POINTS

Multi-Branch Database Architecture & Implementation Specification

Flutter • Existing Firebase Authentication • Existing Cloud Firestore • Drift / SQLite • Cloud Functions

Critical authentication compatibility decision
Existing users who currently log in with mobile number + PIN must continue using that same login method. Do not force them to create email/password credentials. Only newly created users will use email + password login.

Document detail	Specification
Product	Balaji Points
Firebase project	Existing project: balajipoints (retain)
Application	Flutter – Android, iOS, tablet and desktop; web storage adapter if web is supported
Architecture	Local-first data access with controlled cloud synchronization
Prepared for	Aaiji Tech Studio
Version	1.0 – 02 November 2026

Implementation baseline. Existing production collections and authentication flows must be audited before any migration or deployment.

1. Executive Summary

Balaji Points will operate as a centralized multi-branch hardware-store rewards platform. The existing Firebase project and Firestore database will be retained. Drift/SQLite will be added as the local read/cache layer, while Cloud Functions will enforce sensitive business operations such as bill approval, points credit, redemption, branch administration and database cleanup.

The application will have one central Super Admin, up to two active admins per branch, and branch users in four categories: Carpenter, Architect, Interior Designer and Engineer. Every user is assigned to exactly one branch. Branch transfers are not permitted.

Most important: preserve the current login experience for existing users. Authentication migration must not lock out existing users.

2. Finalized Business Rules

ID	Rule	Final decision
BR-01	Super Admin	One central Super Admin role; privileged provisioning only.
BR-02	Branch creation	Super Admin only.
BR-03	Branch admins	Maximum two active admins per branch.
BR-04	Branch users	Carpenter, Architect, Interior Designer, Engineer.
BR-05	Branch assignment	Each account belongs to exactly one branch.
BR-06	Branch transfer	Never allowed. Deactivate old account; create a separate account for another branch if needed.
BR-07	Branch admin access	Own branch data only.
BR-08	Bill upload	Any active branch user can upload their bill.
BR-09	Bill approval	Either of the assigned branch admins can approve or reject.
BR-10	Points credit	Immediately after one admin successfully approves the bill.
BR-11	Duplicate approval	A bill can award points only once.
BR-12	Wallet	Backend-controlled; never trust local balance for redemption.
BR-13	Cleanup	Super Admin only; auditable and recoverable.

3. Authentication Compatibility – Existing and New Users

3.1 Existing users: keep mobile number + PIN

All users who already use mobile number + PIN must continue to log in using the same method. The new architecture must not force these users to set an email address or password. Their existing PIN verification and session behavior must be preserved while the database and synchronization layers are upgraded.

- Do not bulk-reset existing PINs.
- Do not require email/password enrollment for existing accounts.
- Do not delete or recreate existing Firebase Auth users during the database migration.
- Keep the existing phone + PIN verification flow and its current security controls.
- Map each existing account to a stable Firebase Auth UID and preserve its current user profile and branch association.

- If the current PIN system is custom rather than a Firebase Auth provider, retain its trusted verification backend. Do not store or compare plain-text PINs in Flutter or Firestore.

3.2 New users: email + password

Only newly created users will use email + password login. New account provisioning must create a Firebase Authentication email/password account and a corresponding Firestore user profile. The branch and role are assigned by the trusted backend based on the creating administrator's permissions.

```
New user creation flow
Branch Admin / Super Admin
|
v
Select user category and enter profile
|
v
Trusted backend validates creator and branch
|
v
Create Firebase Auth email/password account
|
v
Create users/{firebaseUid} profile
|
v
Create wallets/{firebaseUid}
|
v
Increment branch userCount atomically
|
v
Return account creation result
```

3.3 Authentication profile fields

Field	Existing user	New user
uid	Preserve existing Auth UID	New Firebase Auth UID
loginMethod	legacy_phone_pin	email_password
phone	Preserve existing phone	Required if business policy requires it
email	Optional / may be absent	Required and verified according to policy
role	Mapped without privilege escalation	Assigned by backend
branchId	Preserve verified existing branch	Assigned by backend
status	Preserve / validate	active after provisioning

Suggested profile field: loginMethod = legacy_phone_pin or email_password. This is metadata for account routing and support; it is not a substitute for verifying the actual Firebase Auth identity.

3.4 Important security and migration note

A mobile number + PIN is not automatically a native Firebase Authentication provider. If the existing app uses a custom PIN system, the existing trusted verification service must remain in place. It must securely validate the PIN and issue or maintain a Firebase-authenticated session. Do not implement a client-only PIN check. If the current flow already exchanges credentials for a Firebase custom token, preserve that integration.

Do not assume that a user's email/password can be added to an existing UID without checking Firebase Auth provider-linking support and the current identity model. The requirement here is to keep existing users on their existing method; new users use email/password.

4. Roles and Access Control

Capability	Super Admin	Branch Admin	Branch User
View all branches	Yes	No	No
Create / deactivate branch	Yes	No	No
Assign branch admins	Yes	No	No
Create users	All branches	Own branch	No
View users	All	Own branch	Own profile
Upload bill	No	No	Yes
Approve / reject bill	All branches	Own branch	No
View wallets	All	Own branch	Own wallet
Redeem points	Authorized	Authorized	Own wallet
Reset points / archive	Yes	No	No
View audit logs	All	Own branch where allowed	No
Change branch assignment	No transfer allowed	No	No

Use separate role and userType fields. role values: super_admin, branch_admin, branch_user. userType values for branch users: carpenter, architect, interior_designer, engineer. Branch admins have userType = null.

Enforce one active Super Admin through a protected provisioning process and singleton configuration. Do not expose Super Admin creation in normal user-management screens.

5. Target Firestore Collection Structure

```
balaji_points (existing Firebase project)
|
|-- branches/{branchId} [new or extend existing]
|-- users/{firebaseUid} [existing; normalize carefully]
|-- branch_admins/{firebaseUid} [new if no equivalent exists]
|-- wallets/{firebaseUid} [new if no equivalent exists]
|-- bills/{billId} [existing; extend carefully]
|-- points_history/{transactionId} [existing; preserve history]
|-- notifications/{notificationId}
|-- audit_logs/{logId}
|-- system_settings/{settingId}
|-- sync_metadata/{syncId}
|-- branch_stats/{branchId} [derived dashboard aggregates]
|-- admin_jobs/{jobId} [long-running reset/archive jobs]
```

5.1 Branch document

```
{
"branchId": "BR_VAD_001",
"branchName": "Vadodara Branch",
"branchCode": "VAD001",
"status": "active",
"userLimit": 1000,
"userCount": 245,
"adminCount": 2,
"createdAt": "SERVER_TIMESTAMP",
"createdBy": "SUPER_ADMIN_UID",
"updatedAt": "SERVER_TIMESTAMP",
"schemaVersion": 1
}
```

5.2 User document

```
{
"uid": "FIREBASE_AUTH_UID",
"name": "Sample User",
"phone": "EXISTING_OR_NEW_PHONE",
"email": "OPTIONAL_FOR_LEGACY_REQUIRED_FOR_NEW",
"loginMethod": "legacy_phone_pin",
"role": "branch_user",
"userType": "carpenter",
"branchId": "BR_VAD_001",
"status": "active",
"createdAt": "SERVER_TIMESTAMP",
"createdBy": "ADMIN_UID",
"schemaVersion": 2
}
```

5.3 Wallet, bill and points-history documents

```
wallets/{firebaseUid}
{
"userId": "USER_UID",
"branchId": "BR_VAD_001",
"balance": 1250,
"lifetimeEarned": 1800,
"lifetimeRedeemed": 550,
"version": 12,
"updatedAt": "SERVER_TIMESTAMP"
}

bills/{billId}
{
"userId": "USER_UID",
"branchId": "BR_VAD_001",
"billNumber": "INV-10025",
"amount": 15000,
"imageUrls": ["STORAGE_PATH"],
"status": "pending",
"submittedAt": "SERVER_TIMESTAMP",
"reviewedBy": null,
"reviewedAt": null,
"pointsAwarded": 0,
"approvalTransactionId": null
}

points_history/{transactionId}
{
"userId": "USER_UID",
"branchId": "BR_VAD_001",
"type": "bill_earned",
"points": 150,
"balanceBefore": 1100,
"balanceAfter": 1250,
"referenceId": "BILL_ID",
"idempotencyKey": "APPROVE_BILL_ID",
"createdBy": "ADMIN_UID",
"createdAt": "SERVER_TIMESTAMP"
}
```

Document IDs should use Firebase Auth UID for users and wallets. Existing legacy user documents may use phone numbers or other IDs; audit and map them before normalizing. Never overwrite an existing record based only on a phone-number match.

6. Bill Upload and Approval Workflow

```
Branch user
|
v
Capture bill details and image
|
v
Upload image to Firebase Storage
|
v
Submit bill through trusted backend
|
v
Bill status = pending
|
v
Either assigned branch admin reviews
|
+------ Approve ----------------------+
| |
| Atomic Firestore transaction
| - verify admin and branch
| - verify bill is pending
| - check idempotency
| - calculate eligible points
| - update wallet
| - create points_history
| - mark bill approved
| - create audit record
| |
| v
| Commit once; notify user
|
+------ Reject -> record reason -> notify user
```

The points conversion formula is not specified in this document. The implementation must reuse the existing Balaji Points points policy if one exists, or require a separate business decision before release. Do not invent a conversion rate.

The approval function must read all required documents before writing within the Firestore transaction. Do not send notifications or perform external side effects inside the transaction callback. Trigger notifications after successful commit, using an idempotent event/outbox mechanism.

Situation	Expected behavior
Admin A approves pending bill	Approve and credit points once.
Admin B approves same bill afterward	Return already processed; no second credit.
Admin approves bill from another branch	Permission denied.
Bill is rejected	No points credited; rejection reason recorded.
Bill submission retried	Return original result using request ID; no duplicate bill.

7. Security Architecture

- Firebase Auth verifies identity; Firestore user profile and trusted membership records determine current role and branch.
- Never trust branchId, role, points, amount, or approval status supplied by a client without backend validation.
- Clients cannot directly write wallets or points_history.
- Branch admins can read only records whose branchId matches their current trusted branch assignment.
- Users can read only their own wallet, bills and points history.
- Super Admin provisioning and destructive operations require re-authentication and audit logging.
- Use Firebase App Check as an additional abuse-reduction layer; it does not replace authorization.

- Cloud Functions using Admin SDK bypass Firestore Rules and must perform their own authorization checks.

7.1 Rules policy

Firestore Security Rules should allow only the intended direct reads and deny client writes to privileged collections. Query constraints must include branchId for branch-admin lists because rules do not filter query results. Test rules in the Firebase Emulator Suite before production deployment.

7.2 Storage policy

```
bill_uploads/{branchId}/{userUid}/{billId}/{fileName}

Permit upload only when:
- requester is authenticated
- user profile is active
- requester UID matches userUid
- branchId matches trusted user profile
- bill is in an uploadable state
- file type and size are allowed

Prevent overwriting original bill evidence after approval.
```

7.3 Audit requirements

Log bill approvals/rejections, points adjustments, redemptions, branch creation/deactivation, admin assignment/removal, account deactivation, reset/archive jobs and permission-sensitive changes. Audit records should include actor UID, branch scope, action, target ID, reason, timestamp and request/job ID.

8. Local Database – Drift / SQLite

Each device has a local database. Drift is the Flutter data-access layer. The local database provides fast rendering and cached reads; it is not the authority for roles, bill approval or confirmed wallet balances.

Local table	Purpose
local_branches	Cached branch information
local_users	Authorized cached user profiles
local_wallets	Cached wallet balances and cloud version
local_bills	Bill details and local drafts
local_bill_images	Local upload references and image status
local_points_history	Cached transaction history
local_notifications	Cached notifications
local_sync_queue	Pending operations / outbox
local_sync_metadata	Per-collection cursors
local_app_settings	Non-sensitive preferences
local_pending_uploads	Bill image upload state

Use UUIDs for local record IDs, retain cloud document IDs separately, use integer point values, add indexes for branch/user/date/status filters, and paginate large lists. On logout, stop synchronization and clear or isolate account-specific data so another person using the same device cannot see prior cached data.

8.1 Platform note

Drift with native SQLite is suitable for Android, iOS and desktop. If Flutter Web is a target, implement a web-compatible Drift storage adapter (for example SQLite/WASM with persistent browser storage) and test browser persistence separately. Do not assume native SQLite behavior on the web.

8.2 Avoid competing sync engines

Firestore has its own offline persistence on supported clients. Because the application will use Drift as its UI-local source of truth, implement one explicit synchronization pipeline. Do not let Firestore offline writes and a separate custom outbox independently retry the same financial operation.

9. Synchronization Specification

Role	Local data scope
Super Admin	Branch summaries; paginated users, bills and reports as needed.
Branch Admin	Own branch users, wallets, bills, transaction summaries and notifications.
Branch User	Own profile, wallet, bills, points history and notifications.

```
Outbox item
{
"id": "LOCAL_UUID",
"operation": "submit_bill",
"entityType": "bill",
"entityId": "BILL_UUID",
"branchId": "BRANCH_ID",
"status": "pending",
"retryCount": 0,
"createdAt": "LOCAL_TIMESTAMP"
}

Statuses:
pending | processing | synced | failed
```

- Login: download only data authorized for the current account.
- Dashboard: render from SQLite immediately, then refresh from cloud.
- Bill draft: save locally while offline.
- Bill submission: queue; upload image and submit when online.
- Approval and points credit: online trusted backend operation only.
- App resume: pull incremental changes using stable server cursor/sequence.
- Wallet, bill status, points history, role and branch membership: cloud authoritative.
- Use one sync worker per account/device, retry backoff, idempotency and server acknowledgements.

Do not rely only on client timestamps for incremental synchronization. Use a stable cursor or server-maintained change sequence to avoid missing records with equal or out-of-order timestamps.

10. Cloud Functions Inventory

Function	Authorization	Purpose
createBranch	Super Admin	Create branch and initialize counters/settings.
updateBranch	Super Admin	Update permitted branch metadata.
createBranchAdmin	Super Admin	Assign admin; enforce max two active admins.
createBranchUser	Super Admin / own Branch Admin	Provision email/password account and profile.
deactivateUser	Super Admin / own Branch Admin	Disable account without deleting history.
submitBill	Branch User	Validate bill and create pending record.
approveBill	Super Admin / own Branch Admin	Atomic approval, wallet credit and ledger.
rejectBill	Super Admin / own Branch Admin	Reject with reason and audit.
redeemPoints	Authorized user	Atomic balance validation and redemption.
adjustPoints	Authorized admin	Controlled adjustment with reason.
resetBranchPoints	Super Admin	Create resumable reset job and ledger adjustments.
archiveBranchData	Super Admin	Archive according to retention policy.
pullChanges	Authenticated scoped user	Return paginated authorized changes.
getGlobalDashboard	Super Admin	Return global aggregate metrics.
getBranchDashboard	Own Branch Admin	Return branch aggregate metrics.

Use TypeScript/Node.js Cloud Functions with Firebase Admin SDK. Every function must verify auth, active account status, role, branch membership, request validation and idempotency where relevant.

11. Super Admin Database Management

Operation	Implementation
Reset points	Create a negative adjustment ledger entry for each affected wallet; do not erase history.
Archive bills	Move or mark records according to a recoverable retention policy.
Archive old history	Preserve auditability and recovery; restrict access.
Deactivate users	Disable access; preserve profile and transaction references.
Rebuild wallets	Reconcile from verified ledger and adjustments.
Deactivate branch	Block new activity; retain historical records.

```
Reset job example
{
"jobId": "RESET_UUID",
"type": "branch_points_reset",
"branchId": "BR_VAD_001",
"status": "processing",
"requestedBy": "SUPER_ADMIN_UID",
"reason": "Approved reset reason",
"processedCount": 0,
"totalCount": 245,
"createdAt": "SERVER_TIMESTAMP"
}
```

Large reset/archive operations must be resumable background jobs, not one oversized Firestore transaction. Require Super Admin re-authentication, reason, impact preview, backup/recovery plan, job progress and audit record. Never expose a general database wipe action.

12. Performance, Indexes and Scale

Area	Implementation
Branch user list	30–50 records per page; indexed branch/status/name.
Bills	20–50 per page; indexed branch/status/date.
Points history	20–50 per page; indexed user/date or branch/date.
Dashboard	Local aggregate cache plus server-side summary documents.
Images	Compressed upload and thumbnail; avoid loading originals in lists.
Sync	Incremental pull with stable cursor and scoped queries.
Wallet	Server transaction; versioned cached copy.
Branch capacity	Default configurable limit of 1,000 users.

Suggested indexes (verify against actual queries and Firestore's generated index errors):

- bills: branchId ASC, status ASC, submittedAt DESC
- bills: userId ASC, submittedAt DESC
- bills: branchId ASC, submittedAt DESC
- points_history: userId ASC, createdAt DESC
- points_history: branchId ASC, createdAt DESC
- users: branchId ASC, status ASC, name ASC
- users: branchId ASC, userType ASC, status ASC
- notifications: userId ASC, createdAt DESC

These are engineering targets, not guaranteed timings: cached dashboard under 500 ms, local lookup under 100 ms, navigation under 200 ms. Measure on representative low- and mid-range devices.

13. Flutter Project Structure

```
lib/
app/
app.dart
router/
theme/
initialization/
core/
database/
network/
sync/
security/
errors/
services/
features/
authentication/
super_admin/
dashboard/
branches/
users/
database_management/
reports/
branch_admin/
dashboard/
users/
bills/
wallets/
reports/
branch_user/
dashboard/
wallet/
upload_bill/
my_bills/
profile/
points/
notifications/
settings/
shared/
widgets/
models/
components/

functions/
src/
middleware/
branches/
users/
bills/
wallets/
sync/
admin/
index.ts
```

Use Feature-First + Clean Architecture. Screens must not call Firestore or SQLite directly. UI reads through Riverpod providers and repositories; the sync engine is independent of presentation; privileged operations go through Cloud Functions.

14. Migration and Release Plan

Phase	Work	Exit condition
1. Audit	Inspect Firestore collections, Auth users, PIN flow, rules, Storage, point policy and balances.	Documented field mapping and verified backup.
2. Compatibility	Preserve existing phone+PIN login; test existing users end-to-end.	Existing users can log in unchanged.
3. New login	Add email/password provisioning for newly created users only.	New accounts authenticate and map to correct UID/profile.
4. Branch model	Add branch records, admin membership, immutable assignment and limits.	Cross-branch tests pass.
5. Secure workflows	Implement bill submission, approval, rejection and wallet ledger.	Atomic and duplicate-safe transactions pass.
6. SQLite	Add Drift schema, indexes, migrations and repositories.	Local reads and data migrations pass.
7. Sync	Implement outbox, pull cursors, retry and cache isolation.	Offline drafts and reconnect behavior pass.
8. Admin controls	Implement reset/archive jobs and audit logs.	Recovery and isolation tests pass.
9. Release	Staging pilot, reconciliation, monitoring and controlled production cutover.	No unexplained balance differences.

Do not switch all production writes at once. Keep existing financial writes stable until the new backend is tested, reconciled and ready for a controlled cutover. Avoid running old and new points-award paths simultaneously.

15. Mandatory Acceptance Tests

Test	Expected result
Existing phone + PIN account logs in	Works with the same login method; no email/password requirement.
New email/password account logs in	Works and maps to its correct Firebase UID.
Existing PIN is unchanged	No bulk reset or forced credential migration.
Branch admin opens another branch	Denied by backend and rules.
User tries to change branchId	Rejected.
Third active branch admin is assigned	Rejected.
Same bill approved by two admins	Points credited once.
Bill from another branch approved	Denied.
Offline bill draft	Saved locally and queued.
Offline redemption	Not permitted.
Branch points reset	Only selected branch affected; ledger retained.
Deactivated user acts	Protected operations denied.
Pending sync after app restart	Queue survives and resumes safely.
Wallet reconciliation	Balance matches ledger and approved adjustments.

16. Suggested Flutter Dependencies

```
flutter pub add drift drift_flutter sqlite3_flutter_libs
flutter pub add flutter_riverpod go_router
flutter pub add firebase_core firebase_auth cloud_firestore
flutter pub add cloud_functions firebase_storage
flutter pub add firebase_app_check
flutter pub add connectivity_plus flutter_secure_storage uuid

flutter pub add --dev drift_dev build_runner
```

Select package versions compatible with the project's current Flutter/Dart SDK and supported platforms. Configure web storage separately if Flutter Web is in scope.

16.1 Build and test

```
flutter doctor -v
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter test

cd functions
npm install
npm run build
cd ..

# Deploy only after emulator/staging validation:
firebase deploy --only functions
firebase deploy --only firestore:rules,firestore:indexes
firebase deploy --only storage
```

Use Firebase Emulator Suite for Auth, Firestore, Functions and Storage tests. Use a staging Firebase project or isolated staging data before production deployment. Never replace current production rules without validating existing login and data access.

17. Items Requiring Existing-Project Audit Before Coding

This architecture is the approved target design. Exact migration scripts and code must be generated only after examining the current project and database.

- Current Firestore collection names, document IDs and exact field types.
- How existing mobile + PIN login verifies the PIN and establishes Firebase Auth identity.
- Whether all existing users have stable Firebase Auth UIDs and how phone-ID documents map to them.
- Current branch information and how existing users should be assigned where branchId is missing.
- Current points earning conversion formula and redemption rules.
- Current bill image Storage paths and approval fields.
- Existing Firestore and Storage security rules.
- Current wallet balances reconciled against points history.

Do not infer missing branch assignments or merge accounts solely by matching phone numbers. Resolve identity and branch mapping explicitly.

18. Final Architecture Decision

Component	Final decision
Firebase project	Keep existing balajipoints project.
Existing authentication	Keep mobile number + PIN for all current users.
New user authentication	Email + password for newly created users only.
Cloud database	Keep existing Firestore and extend safely.
Local database	Add Drift + SQLite.
Backend	TypeScript Cloud Functions.
File storage	Existing Firebase Storage, after rules audit.
Roles	One Super Admin; Branch Admin; Branch User.
Branch admins	Maximum two active per branch.
Branch users	Carpenter, Architect, Interior Designer, Engineer.
Branch transfer	Not allowed.
Bill approval	Either assigned branch admin.
Points credit	Immediately after successful approval.
Wallet authority	Trusted backend / Firestore.
Sync	Local-first reads with explicit controlled cloud sync.
Cleanup	Super Admin-only, auditable, recoverable jobs.

Recommended first development task: audit the existing Firebase Auth + phone/PIN implementation and Firestore schema. Preserve current users and balances before implementing new-user email/password login or changing production data.

19. Technical References

Firebase Authentication: https://firebase.google.com/docs/auth

Firebase custom authentication: https://firebase.google.com/docs/auth/admin/create-custom-tokens

Cloud Firestore transactions: https://firebase.google.com/docs/firestore/manage-data/transactions

Firestore Security Rules: https://firebase.google.com/docs/firestore/security/get-started

Firebase App Check: https://firebase.google.com/docs/app-check

Cloud Functions for Firebase: https://firebase.google.com/docs/functions

Flutter database architecture: https://docs.flutter.dev/app-architecture

Drift documentation: https://drift.simonbinder.eu/